Conservatoire Virtuel – Music School Management System

Context

A private music school called **Conservatoire Virtuel** wishes to build its information system. The objective is to design and implement an application to support the school's internal activities.

The system must manage:

- Students and teachers,
- Course packages and individual lessons,
- Lesson scheduling and room/instrument booking,
- Payments and billing,
- Official exams and their results.

The system should follow good software and database design practices.

Functional Requirements

1. Students

A student is identified by a unique ID.
Attributes include:

- Last name, first name,
- Address, date of birth, phone, email,
- Level (Beginner/Intermediate/Advanced),
- Preferred instruments.

Students can enroll in:

- A **Course Package** (N lessons and validity dates),
- Individual lessons (billed per lesson).

The system must keep track of:

- Purchased hours,
- Remaining hours,
- Usage history.

2. Teachers

Each teacher has an ID, name, specializations, hourly rate, qualifications and availability.

A teacher can teach multiple instruments. Scheduling must validate specialization and availability.

3. Course Packages & Services

The school offers:

- Music packages,
- Unlimited lessons packages,
- Group or individual lessons,
- Single paid lessons,
- Instrument rental,
- Room booking.

Each purchased service must be tracked and billable.

4. Scheduling System

The system must allow scheduling of:

- Lessons (teacher + student + room + instrument),
- Room rentals,
- Exam sessions.

For each scheduled activity we keep:

- Date and start time,
- Duration,
- Assigned resources,
- Status (scheduled/completed/cancelled).

Schedule conflicts must be prevented.

5. Exams

The school organizes official exams:

- Exams have a name, instrument, date, and capacity.
- Students can register.
- Results are later recorded: pass/fail and optional score.

6. Payments and Billing

The school must keep track of:

- Payments,
- Outstanding balances,
- Prices of packages, rentals and lessons.

The system must provide:

- Total amount spent by a student,
- Summary per month or per student.

Additional Business Rules

1. Package hours expire at the package end date.
2. If a lesson is cancelled less than 24h before, it counts as consumed.
3. Exam registration closes when maximum capacity is reached.
4. A group lesson consumes one hour from each participant.

Object-Oriented Design Requirements (mandatory)

Your design and implementation must apply advanced OOP concepts.

1. Abstract Classes

The solution must include at least two abstract classes used appropriately to model generalization and shared behavior.

2. Interfaces

At least two interfaces must be used to represent independent capabilities or behaviors.

3. Polymorphism

You must demonstrate the use of:

- Polymorphic collections,
- Polymorphic method calls,
- Overridden methods and dynamic dispatch.

4. Copy Constructors

You must implement copy constructors for multiple key classes to support duplication and protect internal state.

5. Good OOP practice

Your design must show correct use of:

- Inheritance,
- Composition,
- Encapsulation,
- Abstract types versus concrete types.

These constraints must appear in:

- UML diagrams,
- Java code,
- Final report description.

Deliverables

1. UML Design

Include:

- Class Diagram,
- Sequence Diagram for scheduling,
- Activity Diagram for exam registration.

2. Java Application

Create a console-based application with a text menu supporting:

a. Manage students and teachers
b. Manage course packages and individual lessons
c. Manage scheduling and booking
d. Manage payments and billing
e. Manage exams and results

The application must contain enough test data to demonstrate functionalities.

3. Use Case Preparation

Prepare a scenario with initial data:

- Students, teachers,
- Packages,
- Instruments,
- Rooms,
- Scheduled lessons,
- Exams.

4. Report + Presentation

Your report must contain:

- Project description,
- UML,
- Design rationale,
- Screenshots of execution,
- Testing scenario,
- Discussion of OO requirements.

5. Bonus (optional)

- Implement a database,
- Add a GUI (Swing/JavaFX),
- Add unit tests,

Grading Criteria

- Correctness of domain implementation,
- Code quality and object-oriented design,
- Correct use of abstract classes, interfaces, polymorphism and copy constructors,
- Professional UML,
- Clarity and organization of report and presentation.